

In [1]:

```
import numpy as np

def echelon_form(matrix):
    """
    Convert the input matrix to echelon form using Gaussian elimination.

    Parameters:
    matrix (numpy.ndarray): The input matrix to be converted to echelon form.

    Returns:
    numpy.ndarray: The matrix in echelon form.
    """
    m, n = matrix.shape
    current_row = 0

    for col in range(n):
        if current_row >= m:
            break

        # Find the first non-zero pivot element in the column
        pivot_row = current_row
        while pivot_row < m and matrix[pivot_row, col] == 0:
            pivot_row += 1

        if pivot_row == m:
            continue

        # Swap the current row with the pivot row
        matrix[[current_row, pivot_row]] = matrix[[pivot_row, current_row]]

        # Scale the pivot row so that the pivot element becomes 1
        pivot_element = matrix[current_row, col]
        matrix[current_row] /= pivot_element

        # Eliminate values below the pivot element
        for row in range(current_row + 1, m):
            factor = matrix[row, col]
            matrix[row] -= factor * matrix[current_row]

        current_row += 1

    return matrix

# Example matrix
input_matrix = np.array([[2, 1, -1, 8],
                          [-3, -1, 2, -11],
                          [-2, 1, 2, -3]], dtype=float) # Specify dtype as float

print("The input Matrix is:- ")
print(input_matrix)
print("-"*100)
result = echelon_form(input_matrix)
print("Echelon Form:")
print(result)
```

```
The input Matrix is:-
[[ 2.  1. -1.  8.]
 [-3. -1.  2. -11.]
 [-2.  1.  2. -3.]]
```

```
-----
Echelon Form:
[[ 1.  0.5 -0.5  4. ]
 [ 0.  1.  1.  2. ]
 [-0. -0.  1. -1. ]]
```


Gaussian Elimination

In [1]:

```
import numpy as np

def gaussian_elimination_pivot(A, b):
    # Combine the matrix A and vector b into one augmented matrix
    augmented_matrix = np.column_stack((A, b))

    n = len(b)

    for k in range(n):
        # Partial pivoting: find the pivot (maximum element in the current column)
        pivot_index = np.argmax(np.abs(augmented_matrix[k:, k])) + k

        # Swap rows to move the pivot element to the current row
        augmented_matrix[[k, pivot_index]] = augmented_matrix[[pivot_index, k]]

        # Eliminate entries below the pivot
        for i in range(k + 1, n):
            factor = augmented_matrix[i, k] / augmented_matrix[k, k]
            augmented_matrix[i, k:] -= factor * augmented_matrix[k, k:]

    # Back substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (augmented_matrix[i, -1] - np.dot(augmented_matrix[i,
            i+1:n], x[i+1:])) / augmented_matrix[i, i]

    return x
```

In [2]:

```
# Example usage
A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]]).astype(np.float64)

b = np.array([8, -11, -3]).astype(np.float64)

print("The augmented matrix is:- ")
print(np.column_stack((A, b)))
print("-"*50)

solution = gaussian_elimination_pivot(A, b)
print("Solution:", solution)
```

The augmented matrix is:-

```
[[ 2.  1. -1.  8.]
 [-3. -1.  2. -11.]
 [-2.  1.  2. -3.]]
```

Solution: [2. 3. -1.]

Gauss-Jordan Elimination Algorithm

In [1]:

```
import numpy as np

def gauss_jordan_elimination(A, b):
    # Combine the matrix A and vector b into one augmented matrix
    augmented_matrix = np.column_stack((A, b))

    n = len(b)

    for k in range(n):
        # Partial pivoting: find the pivot (maximum element in the current column)
        pivot_index = np.argmax(np.abs(augmented_matrix[k:, k])) + k

        # Swap rows to move the pivot element to the current row
        augmented_matrix[[k, pivot_index]] = augmented_matrix[[pivot_index, k]]

        # Make the pivot element 1
        augmented_matrix[k, :] /= augmented_matrix[k, k]

        # Eliminate entries above and below the pivot
        for i in range(n):
            if i != k:
                factor = augmented_matrix[i, k]
                augmented_matrix[i, :] -= factor * augmented_matrix[k, :]

    # Extract the solution
    solution = augmented_matrix[:, -1]

    return solution
```

In [2]:

```
# Example usage
A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]]).astype(np.float64)

b = np.array([8, -11, -3]).astype(np.float64)

print("The augmented matrix is:")
print(np.column_stack((A, b)))
print("-" * 50)

solution = gauss_jordan_elimination(A, b)
print("Solution:", solution)
```

The augmented matrix is:

```
[[ 2.  1. -1.  8.]
 [-3. -1.  2. -11.]
 [-2.  1.  2. -3.]]
```

Solution: [2. 3. -1.]

LU Decomposition and System Solving Algorithm

In [1]:

```
import numpy as np

def lu_decomposition(A):
    n = len(A)
    L = np.eye(n)
    U = np.zeros_like(A, dtype=float)

    for k in range(n):
        U[k, k:] = A[k, k:] - L[k, :k] @ U[:k, k:]
        L[k+1:, k] = (A[k+1:, k] - L[k+1:, :k] @ U[:k, k]) / U[k, k]

    return L, U

def solve_system_with_lu_decomposition(A, b):
    L, U = lu_decomposition(A)

    # Forward Substitution (Ly = b)
    y = np.linalg.solve(L, b)

    # Backward Substitution (Ux = y)
    x = np.linalg.solve(U, y)

    return x
```

In [2]:

```
# Example Usage
A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]]).astype(np.float64)

b = np.array([8, -11, -3]).astype(np.float64)

solution = solve_system_with_lu_decomposition(A, b)
print("Solution:", solution)
```

Solution: [2. 3. -1.]

Cholesky Decomposition and System Solving Algorithm

In [1]:

```
import numpy as np

def cholesky_decomposition(A):
    n = len(A)
    L = np.zeros_like(A, dtype=float)

    for i in range(n):
        for j in range(i+1):
            if i == j:
                L[i, i] = np.sqrt(A[i, i] - np.sum(L[i, :i]**2))
            else:
                L[i, j] = (A[i, j] - np.sum(L[i, :i] * L[j, :i])) / L[j, j]

    return L

def solve_system_with_cholesky_decomposition(A, b):
    L = cholesky_decomposition(A)

    # Solve Ly = b using forward substitution
    y = np.linalg.solve(L, b)

    # Solve L^T x = y using backward substitution
    x = np.linalg.solve(L.T, y)

    return x
```

In [2]:

```
# Example Usage
A = np.array([[4, 12, -16],
              [12, 37, -43],
              [-16, -43, 98]]).astype(np.float64)

b = np.array([1, 2, 3]).astype(np.float64)
print("The augmented matrix is:-")
print(np.column_stack((A, b)))
print("-"*50)
solution = solve_system_with_cholesky_decomposition(A, b)
print("The solution is:- ")
print(solution)
```

The augmented matrix is:-

```
[[ 4.  12. -16.  1.]
 [ 12.  37. -43.  2.]
 [-16. -43.  98.  3.]]
```

The solution is:-

```
[28.58333333 -7.66666667  1.33333333]
```

QR Factorization using Gramm Schmidt

In [9]:

```
import numpy as np

def qr_gram_schmidt(A):
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))

    for j in range(n):
        v = A[:, j]
        for i in range(j):
            R[i, j] = np.dot(Q[:, i].T, A[:, j])
            v -= R[i, j] * Q[:, i]
        R[j, j] = np.linalg.norm(v)
        Q[:, j] = v / R[j, j]

    return Q.round(3), R.round(3)
```

In [11]:

```
# Create a matrix A
A = np.array([[1., 2., 0.], [0., 1., 1.], [1., 0., 1.]])

# Perform QR factorization using the Gram-Schmidt process
Q, R = qr_gram_schmidt(A)

# Print the orthogonal matrix Q
print("Orthogonal matrix Q:")
print(Q)
print("***50")
# Print the upper triangular matrix R
print("Upper triangular matrix R:")
print(R)
```

```
Orthogonal matrix Q:
[[ 0.707  0.577 -0.408]
 [ 0.      0.577  0.816]
 [ 0.707 -0.577  0.408]]
*****
```

```
Upper triangular matrix R:
[[1.414 1.414 0.707]
 [0.    1.732 0.    ]
 [0.    0.    1.225]]
```

Finding Dominant Eigenvalue and Eigenvector using Power Method

In [3]:

```
import numpy as np

def power_method(A, num_iterations=1000, tolerance=1e-6):
    n = A.shape[0]

    # Initialize a random vector as an approximation to the dominant eigenvector
    x = np.random.rand(n)
    x /= np.linalg.norm(x)

    for i in range(num_iterations):
        # Perform the power iteration
        x_old = x
        x = np.dot(A, x)

        # Normalize the eigenvector
        x /= np.linalg.norm(x)

        # Check for convergence
        if np.linalg.norm(x - x_old) < tolerance:
            break

    # Calculate the dominant eigenvalue
    eigenvalue = np.dot(x, np.dot(A, x))

    return eigenvalue.round(3), x.round(3)
```

In [4]:

```
# Example usage:
A = np.array([[1., 2., 0.], [2., 1., 0.], [0., 0., -1.]])
dominant_eigenvalue, dominant_eigenvector = power_method(A)

print("Original matrix A:\n", A)
print("\nDominant eigenvalue:", dominant_eigenvalue)
print("Dominant eigenvector:", dominant_eigenvector)
```

Original matrix A:

```
[[ 1.  2.  0.]
 [ 2.  1.  0.]
 [ 0.  0. -1.]]
```

Dominant eigenvalue: 3.0

Dominant eigenvector: [0.707 0.707 -0.]

Singular Value Decomposition

In [11]:

```
import numpy as np

def svd(matrix):
    # Compute U
    AAT = np.dot(matrix, matrix.T)
    eigenvalues_U, eigenvectors_U = np.linalg.eig(AAT)
    normalized_eigenvectors_U = eigenvectors_U / np.linalg.norm(eigenvectors_U, axis=0)
    U = normalized_eigenvectors_U

    # Compute V
    ATA = np.dot(matrix.T, matrix)
    eigenvalues_V, eigenvectors_V = np.linalg.eig(ATA)
    normalized_eigenvectors_V = eigenvectors_V / np.linalg.norm(eigenvectors_V, axis=0)
    V = normalized_eigenvectors_V

    # Compute S
    eigenvalues_S = np.sqrt(np.abs(eigenvalues_U))
    S = np.zeros_like(matrix)
    np.fill_diagonal(S, eigenvalues_S)

    return U.round(3), S.round(3), V.round(3)
```

In [12]:

```
# Example usage
A = np.array([[1., 0., 1., 0.], [0., 1., 0., 1.]])
U, S, V = svd(A)

print("U:")
print(U)
print("*"*50)
print("S:")
print(S)
print("*"*50)
print("V:")
print(V)
```

```
U:
[[1. 0.]
 [0. 1.]]
*****
S:
[[1.414 0.    0.    0.   ]
 [0.    1.414 0.    0.   ]]
*****
V:
[[ 0.707  0.707  0.    0.   ]
 [ 0.    0.   -0.707 -0.707]
 [ 0.707 -0.707  0.    0.   ]
 [ 0.    0.   -0.707  0.707]]
```